Swin Transfomer & ConvNeXt

“Swin Transformer: Hierarchical Vision Transformer using Shifted Windows,” ICCV, 2021.

“A ConvNet for the 2020s,” CVPR, 2022.

박철우

목차

1. Swin Transformer
    -기존 vit의 문제
    -Swin Transformer 구조
    -Swin Transformer 방법
    -실험 결과

2. ConvNeXt
    -제안 배경
    -ResNet 현대화
    -실험 결과
    -코드 입출력 결과

기존 ViT의 문제

1. object detection(객체 탐지)

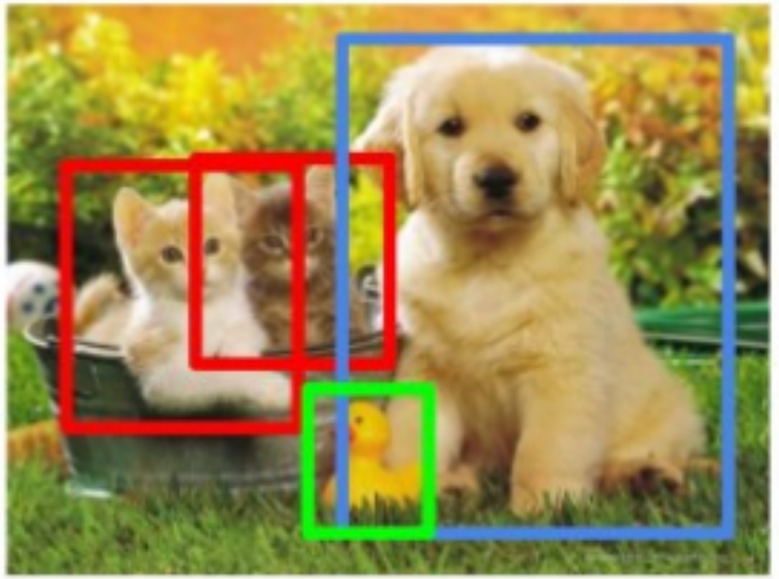
- 단어 토큰과 달리, 이미지 속 시각적 요소는 scale이 다양

- 기존 ViT는 고정 크기 patch token으로 이미지 처리하기에 객체 탐지에 한계

2. Semantic segmentation(의미론적 분할)

- 텍스트 구절의 단어에 비해 이미지의 픽셀 해상도가 훨씬 높음

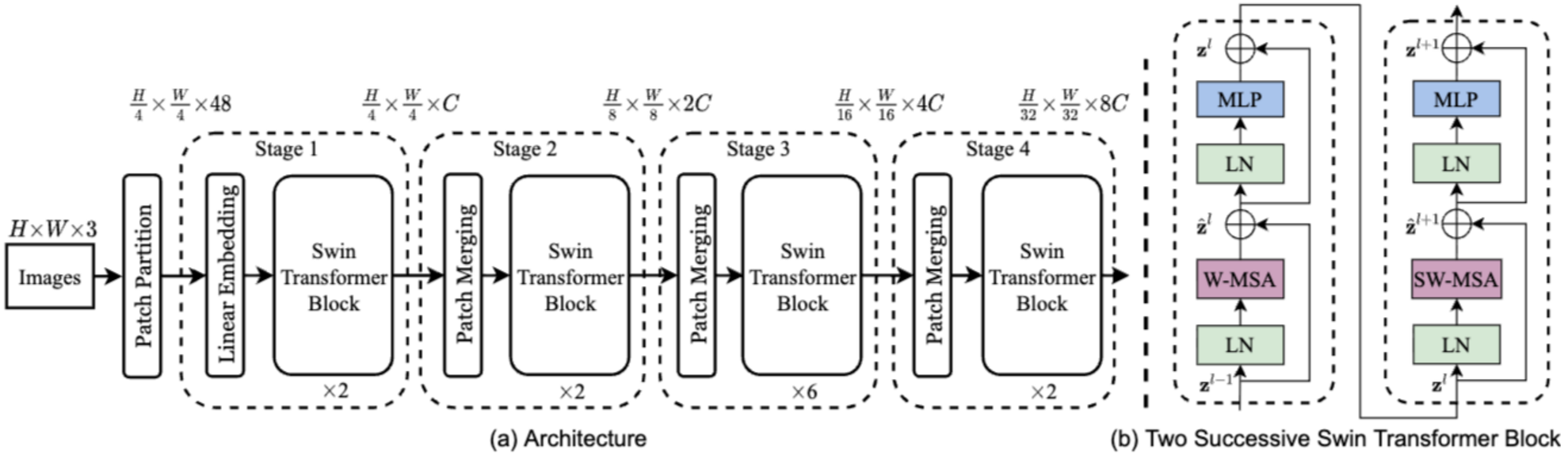
- global self-attention은 이미지 크기에 대해 이차적인 계산 복잡도를 가지므로 고해상도 이미지 작업에 한계



출처 : 위키독스

Swin Transformer

Swin Transformer 구조

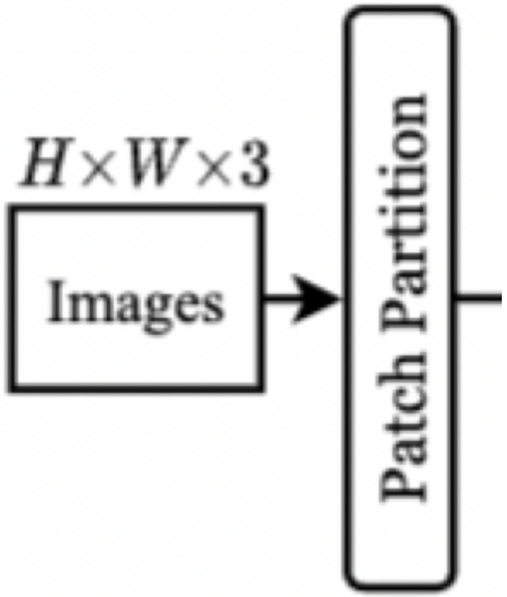


Swin Transformer

Images

- Vit와 같이 패치 분할 모듈을 사용하여 입력 RGB 이미지를 겹치지 않는 패치로 분할

- 이때 전체 토큰 수: H/4 x W/4

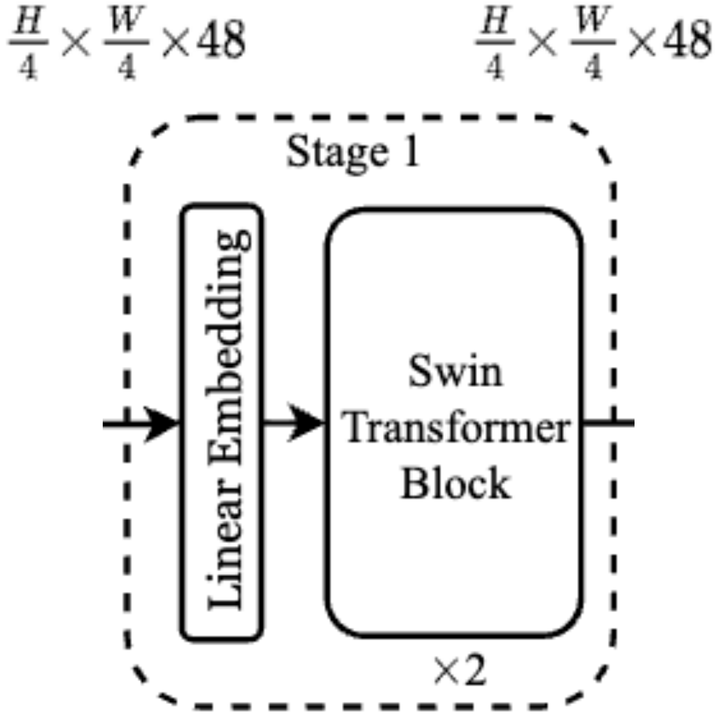


Ex) 이미지를 4x4 크기의 patch로 나누면 patch 하나는 4x4x3 = 48개의 픽셀을 가짐

Swin Transformer

Linear Embedding

- N차원의 patch 벡터 -> C차원 feature 벡터
- 이미지 patch를 Transfomer용 token 벡터로 바꿔주는 역할
- 이때 stage1에서 토큰 수 유지



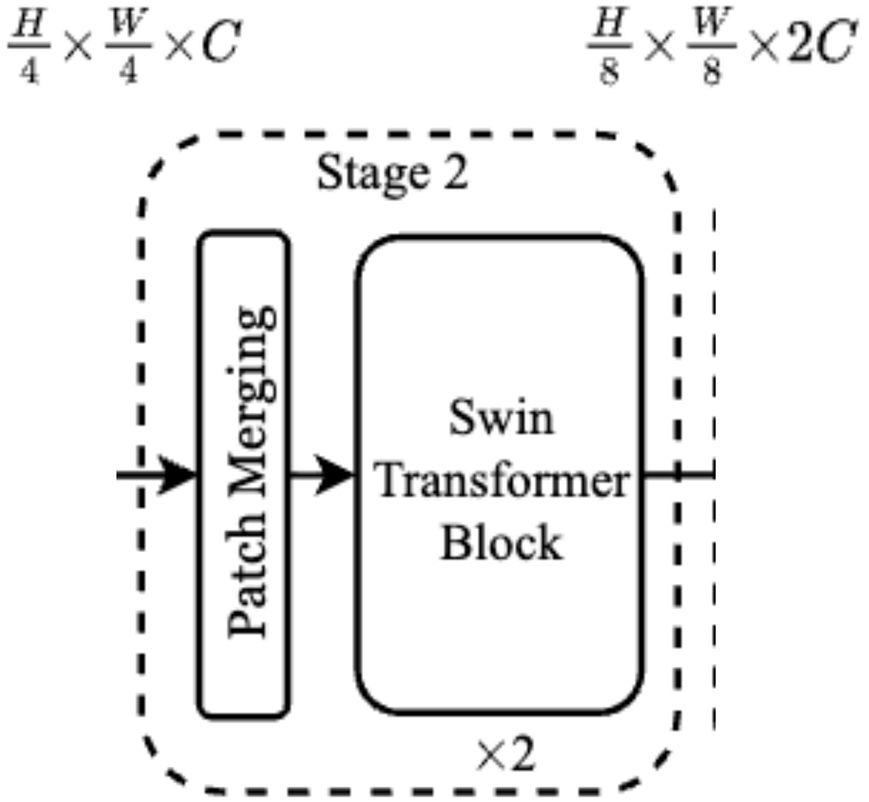
- # 텍스트 transformer에서 단어를 숫자 벡터를 바꾸는 것처럼 swin 에서도 이미지 patch를 숫자 벡터로 바꿔야 줘야함

Ex) 48차원 patch 벡터 -> 96차원 feature 벡터

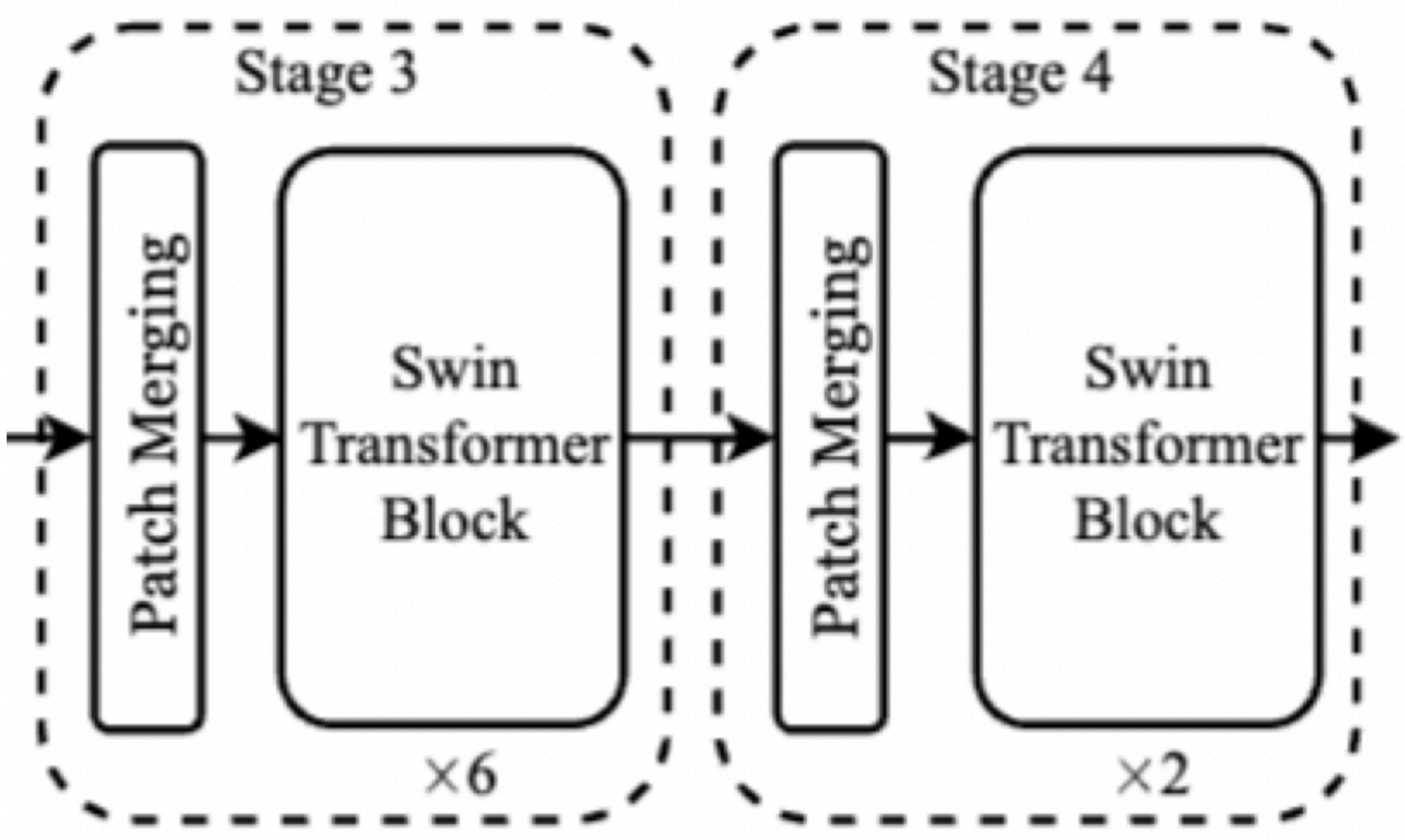
Swin Transformer

Patch Merging

- 주변 2x2 토큰 4개를 하나로 합치는 과정
  -> 4C -> Linear Layer 적용 -> 2C
- 해상도 ↓ : H/4 x W/4 -> H/8 x W/8
- 토큰 수 ↓ : 전체 토큰 수 1/4 감소
- 채널 수 ↑ : C -> 2C



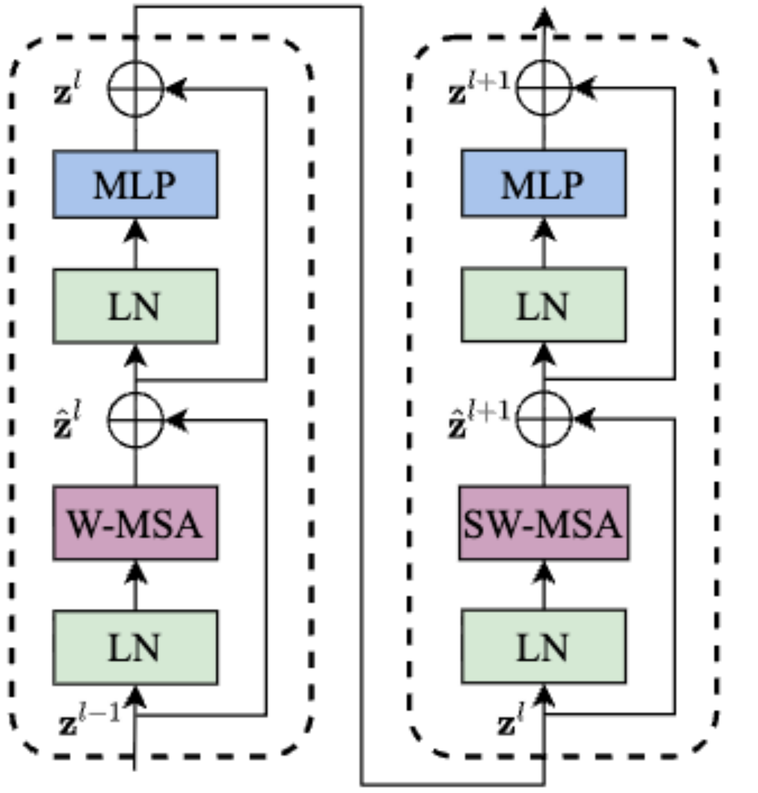
Swin Transformer



Swin Transformer

Swin Transformer Block 구조

- shifted window를 기반으로 하는 multi-head self-attention(MSA) 모듈
- GELU 비선형성을 갖는 2계층 MLP
- MSA 와 MLP 앞에 LayerNorm 레이어 적용

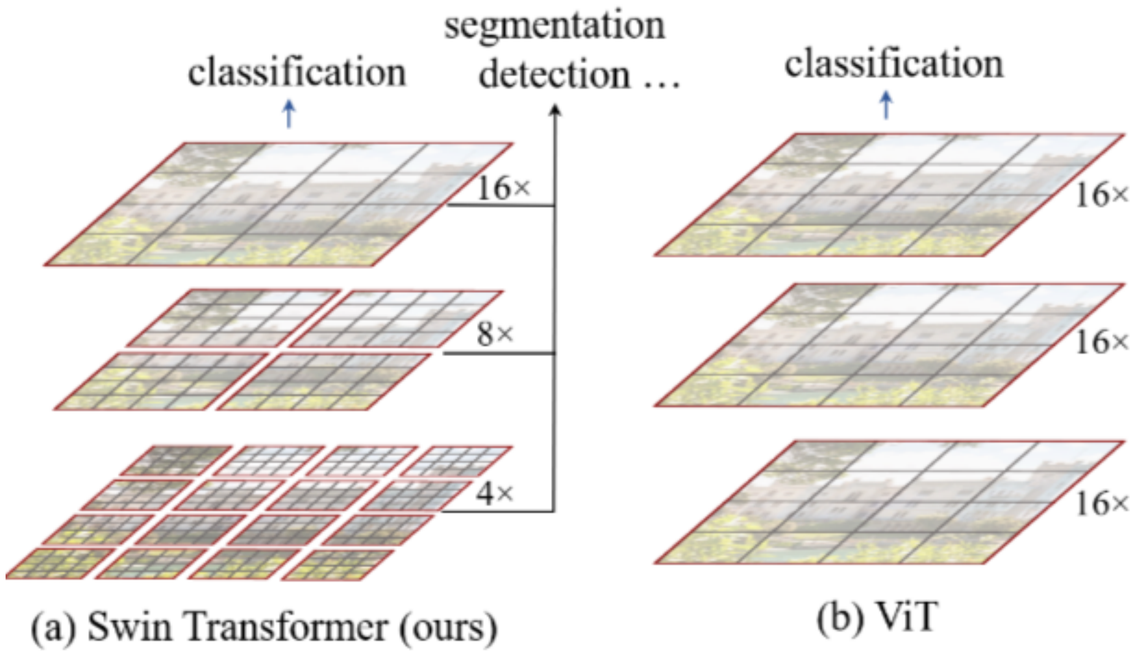


Two Successive Swin Transformer Blocks

Swin Transformer

Swin Transformer 방법

- Window attention
- Shifted window
- Hierarchical structure



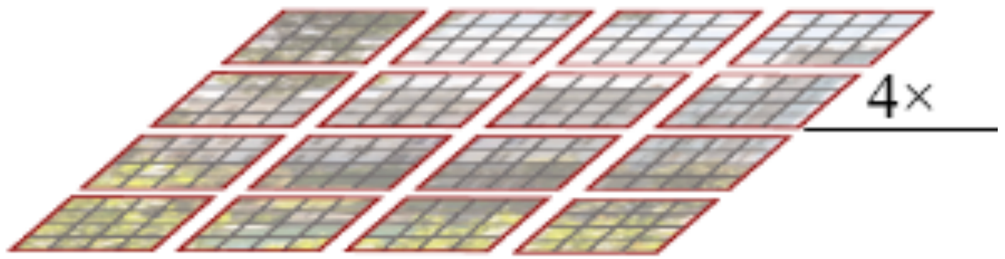
Swin Transformer

Window attention (W-MSA)

- 비겹침 지역 윈도우 내에서 self-attention

$$\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C,$$
$$\Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC$$

- Ex) 각 윈도우가 M x M 패치를 포함한다고 가정,
  h x w 패치의 이미지에 대한 전역 MSA 모듈의 계산 복잡도



Swin Transformer

문제점

다른 Window에 있는 token들과 직접 정보를 주고받기 어려움

Swin Transformer

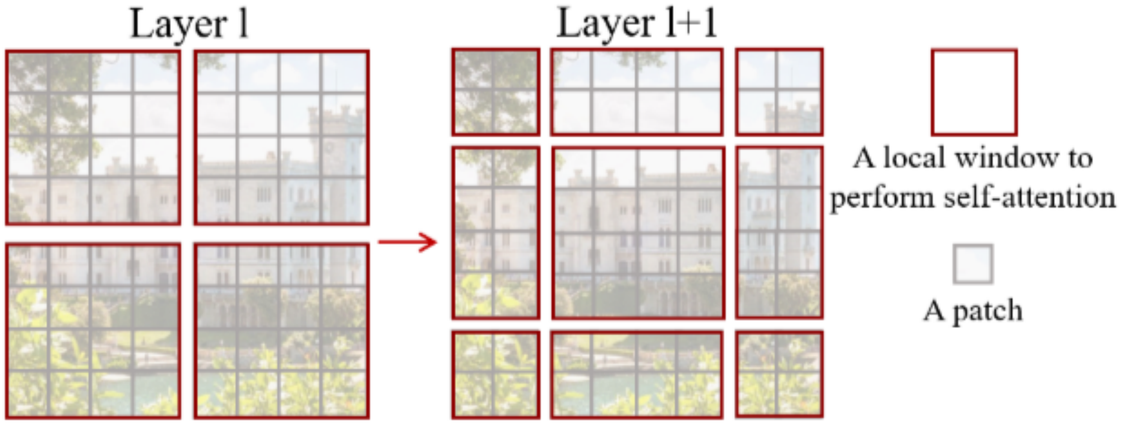
Shifted Window (SW-MSA)

- 윈도우 분할 접근 방식

- window 크기 : M

 - window 구역의 기준을 ([M/2], [M/2]) 이동

이미지 분류, 객체 탐지 및 의미론적 분할에서 효과적



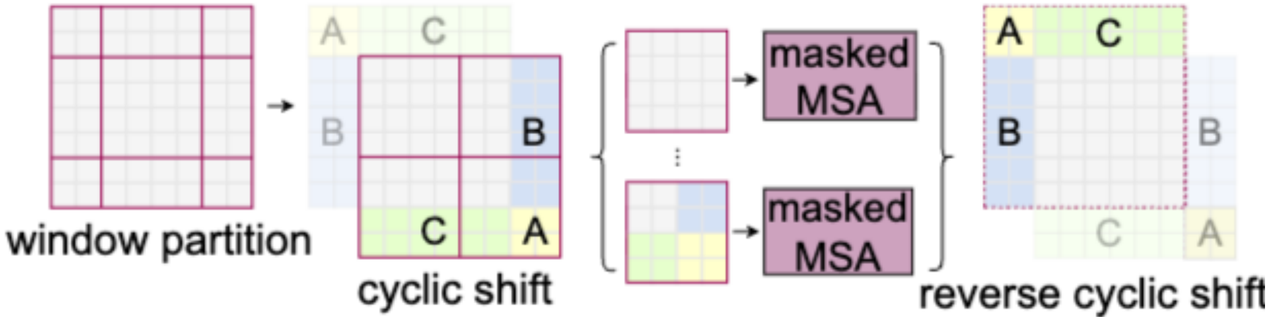
$$
\begin{aligned}
\hat{\mathbf{z}}^{l} &= \text{W-MSA}\left(\text{LN}\left(\mathbf{z}^{l-1}\right)\right) + \mathbf{z}^{l-1}, \\
\mathbf{z}^{l} &= \text{MLP}\left(\text{LN}\left(\hat{\mathbf{z}}^{l}\right)\right) + \hat{\mathbf{z}}^{l}, \\
\hat{\mathbf{z}}^{l+1} &= \text{SW-MSA}\left(\text{LN}\left(\mathbf{z}^{l}\right)\right) + \mathbf{z}^{l}, \\
\mathbf{z}^{l+1} &= \text{MLP}\left(\text{LN}\left(\hat{\mathbf{z}}^{l+1}\right)\right) + \hat{\mathbf{z}}^{l+1},
\end{aligned}
$$

Swin Transformer

Shifted Window (SW-MSA)

- 효율적인 SW-MSA (순환 이동)

- 작은 윈도우를 M x M 크기로 패딩하고 attention 계산 시 padding된 값을 mask 하는 방법



Feature map을 왼쪽 위로 밀고, 밖으로 나간 부분은 반대쪽으로 다시 붙이는 방식

- Mask 하는 이유: 왼쪽 끝과 오른쪽 끝은 실제 이미지에서 이웃이 아니기 때문

Swin Transformer

Relative position bias

- Window 안 token들의 상대적 위치 정보를 반영
- 즉, 두 patch가 서로 얼마나 떨어져 있는지에 따라 attention 점수 보정
- 상대위치 범위: [-M+1, M-1]

장점: 공간적 관계를 더 잘 학습

$$\text{Attention}(Q, K, V) = \text{SoftMax}(QK^T/\sqrt{d} + B)V,$$

$$B \in \mathbb{R}^{M^2 \times M^2}$$

$$\hat{B} \in \mathbb{R}^{(2M-1)\times(2M-1)}.$$

모든 토큰 쌍마다 bias를 따로 만들면 비효율적이기 때문에 사용

Swin Transformer

Experiments (background)

- ImageNet-1K

 - 1000개의 class로부터 1.28M개의 훈련 이미지와 50K개의 검증 이미지를 포함하는 데이터 셋

- ImageNet-22k

- 1420만 개의 이미지와 22,000개의 class를 포함하는 데이터 셋

- Swin-T: $C = 96$, layer numbers = $\{2, 2, 6, 2\}$
- Swin-S: $C = 96$, layer numbers =$\{2, 2, 18, 2\}$
- Swin-B: $C = 128$, layer numbers =$\{2, 2, 18, 2\}$
- Swin-L: $C = 192$, layer numbers =$\{2, 2, 18, 2\}$

Architecture Variants

Swin Transformer

Experiments (background)

- Method : 탐지 모델 이름
- Backbond : method 안에서 이미지 특징을 뽑는 네트워크
- Param : 모델의 총 파라미터 수
- FLOPS : 연산량 (계산 복잡도)
- Throughput (image/s) : 초당 처리할 수 있는 이미지 수
- ImageNet top-1 acc. : ImageNet-1k 데이터 셋에서 모델이 예측한 가장 높은 확률의 클래스

(a) Regular ImageNet-1K trained models

method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
RegNetY-4G [44]	224^2	21M	4.0G	1156.7	80.0
RegNetY-8G [44]	224^2	39M	8.0G	591.6	81.7
RegNetY-16G [44]	224^2	84M	16.0G	334.7	82.9
ViT-B/16 [19]	384^2	86M	55.4G	85.9	77.9
ViT-L/16 [19]	384^2	307M	190.7G	27.3	76.5
DeiT-S [57]	224^2	22M	4.6G	940.4	79.8
DeiT-B [57]	224^2	86M	17.5G	292.3	81.8
DeiT-B [57]	384^2	86M	55.4G	85.9	83.1
Swin-T	224^2	29M	4.5G	755.2	81.3
Swin-S	224^2	50M	8.7G	436.9	83.0
Swin-B	224^2	88M	15.4G	278.1	83.5
Swin-B	384^2	88M	47.0G	84.7	84.5

(b) ImageNet-22K pre-trained models

method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
R-101x3 [34]	384^2	388M	204.6G	-	84.4
R-152x4 [34]	480^2	937M	840.5G	-	85.4
ViT-B/16 [19]	384^2	86M	55.4G	85.9	84.0
ViT-L/16 [19]	384^2	307M	190.7G	27.3	85.2
Swin-B	224^2	88M	15.4G	278.1	85.2
Swin-B	384^2	88M	47.0G	84.7	86.4
Swin-L	384^2	197M	103.9G	42.1	87.3

Swin Transformer

Object Detection on COCO

- (a) : Swin-T 와 ResNet-50의 결과
- (b) : Cascade Mask RCNN 사용
- (c) : 이전 최첨단 모델들과의 비교

(a) Various frameworks

Method	Backbone	AP^{box}	AP^{box}_{50}	AP^{box}_{75}	#param.	FLOPs	FPS
Cascade Mask R-CNN	R-50	46.3	64.3	50.5	82M	739G	18.0
	Swin-T	**50.5**	**69.3**	**54.9**	86M	745G	15.3
ATSS	R-50	43.5	61.9	47.0	32M	205G	28.3
	Swin-T	**47.2**	**66.5**	**51.3**	36M	215G	22.3
RepPointsV2	R-50	46.5	64.6	50.3	42M	274G	13.6
	Swin-T	**50.0**	**68.5**	**54.2**	45M	283G	12.0
Sparse R-CNN	R-50	44.5	63.4	48.2	106M	166G	21.0
	Swin-T	**47.9**	**67.3**	**52.3**	110M	172G	18.4

(b) Various backbones w. Cascade Mask R-CNN

	AP^{box}	AP^{box}_{50}	AP^{box}_{75}	AP^{mask}	AP^{mask}_{50}	AP^{mask}_{75}	param	FLOPs	FPS
DeiT-S†	48.0	67.2	51.7	41.4	64.2	44.3	80M	889G	10.4
R50	46.3	64.3	50.5	40.1	61.7	43.4	82M	739G	18.0
Swin-T	**50.5**	**69.3**	**54.9**	**43.7**	**66.6**	**47.1**	86M	745G	15.3
X101-32	48.1	66.5	52.4	41.6	63.9	45.2	101M	819G	12.8
Swin-S	**51.8**	**70.4**	**56.3**	**44.7**	**67.9**	**48.5**	107M	838G	12.0
X101-64	48.3	66.4	52.3	41.7	64.0	45.1	140M	972G	10.4
Swin-B	**51.9**	**70.9**	**56.5**	**45.0**	**68.4**	**48.7**	145M	982G	11.6

(c) System-level Comparison

Method	mini-val		test-dev		#param.	FLOPs
	AP^{box}	AP^{mask}	AP^{box}	AP^{mask}		
RepPointsV2* [12]	-	-	52.1	-	-	-
GCNet* [7]	51.8	44.7	52.3	45.4	-	1041G
RelationNet++* [13]	-	-	52.7	-	-	-
DetectoRS* [42]	-	-	55.7	48.5	-	-
YOLOv4 P7* [4]	-	-	55.8	-	-	-
Copy-paste [23]	55.9	47.2	56.0	47.4	185M	1440G
X101-64 (HTC++)	52.3	46.0	-	-	155M	1033G
Swin-B (HTC++)	56.4	49.1	-	-	160M	1043G
Swin-L (HTC++)	57.1	49.5	57.7	50.2	284M	1470G
Swin-L (HTC++)*	**58.0**	**50.4**	**58.7**	**51.1**	284M	-

Object Detection,
Instance Segmentation

Swin Transformer

Semantic Segmentation on ADE20K

- 다양한 method / backbone 사용하여 성능 확인
- ImageNet-22k 사전 훈련을 사용한 Swin-L이 가장 높았음

ADE20K Method	Backbone	val mIoU	test score	#param.	FLOPs	FPS
DLab.v3+ [11]	ResNet-101	44.1	-	63M	1021G	16.0
DNL [65]	ResNet-101	46.0	56.2	69M	1249G	14.8
OCRNet [67]	ResNet-101	45.3	56.0	56M	923G	19.3
UperNet [63]	ResNet-101	44.9	-	86M	1029G	20.1
OCRNet [67]	HRNet-w48	45.7	-	71M	664G	12.5
DLab.v3+ [11]	ResNeSt-101	46.9	55.1	66M	1051G	11.9
DLab.v3+ [11]	ResNeSt-200	48.4	-	88M	1381G	8.1
SETR [73]	T-Large‡	50.3	61.7	308M	-	-
UperNet	DeiT-S†	44.0	-	52M	1099G	16.2
UperNet	Swin-T	46.1	-	60M	945G	18.5
UperNet	Swin-S	49.3	-	81M	1038G	15.2
UperNet	Swin-B‡	51.6	-	121M	1841G	8.7
UperNet	Swin-L‡	**53.5**	**62.8**	234M	3230G	6.2

Swin Transformer

Ablation Study

- Shifted window 방식 있을때와 없을때
- No pos 일때, absolute position embedding 사용할 때, relative postion bias 사용할 때로 나눔

	ImageNet		COCO		ADE20k
	top-1	top-5	AP^{box}	AP^{mask}	mIoU
w/o shifting	80.2	95.1	47.7	41.5	43.3
shifted windows	**81.3**	**95.6**	**50.5**	**43.7**	**46.1**
no pos.	80.1	94.9	49.2	42.6	43.8
abs. pos.	80.5	95.2	49.0	42.4	43.2
abs.+rel. pos.	81.3	95.6	50.2	43.4	44.0
rel. pos. w/o app.	79.3	94.7	48.2	41.9	44.1
rel. pos.	**81.3**	**95.6**	**50.5**	**43.7**	**46.1**

Swin Transformer

Different self-attention methods

- Shifted window 방식을 사용하면서 순환 구조를 사용 했을 때 가장 좋은 성능을 보였다.

method	MSA in a stage (ms)				Arch. (FPS)		
	S1	S2	S3	S4	T	S	B
sliding window (naive)	122.5	38.3	12.1	7.6	183	109	77
sliding window (kernel)	7.6	4.7	2.7	1.8	488	283	187
Performer [14]	4.8	2.8	1.8	1.5	638	370	241
window (w/o shifting)	2.8	1.7	1.2	0.9	770	444	280
shifted window (padding)	3.3	2.3	1.9	2.2	670	371	236
shifted window (cyclic)	3.0	1.9	1.3	1.0	755	437	278

Swin Transformer

Different methods on Swin-T

- Swin-T를 sliding window, Performer, shifted window method에서 각각 성능 비교를 하였을 때, shifted window 에서 가장 좋은 성능을 보임

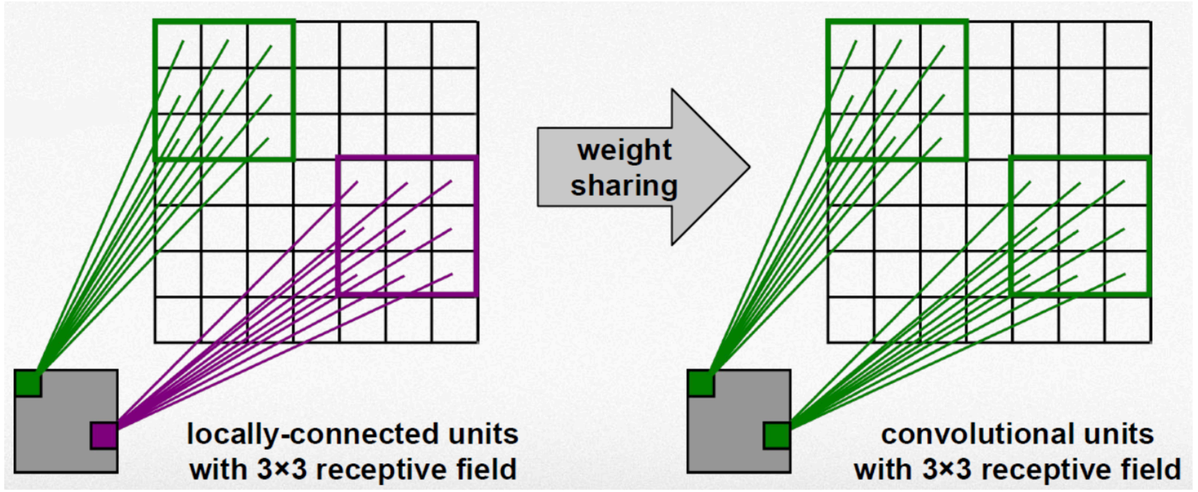
	Backbone	ImageNet top-1	ImageNet top-5	COCO AP^{box}	COCO AP^{mask}	ADE20k mIoU
sliding window	Swin-T	81.4	95.6	50.2	43.5	45.8
Performer [14]	Swin-T	79.0	94.2	-	-	-
shifted window	Swin-T	81.3	95.6	50.5	43.7	46.1

ConvNeXt

Background

CNN의 귀납적 편향

- Locality : 가까운 픽셀끼리 관련
- Weight Sharing : 같은 필터를 이미지 전체에 적용
- Hierarchical Feature : 저수준 특징에서 고수준 특징으로 점진적 학습



ConvNeXt

제안 배경

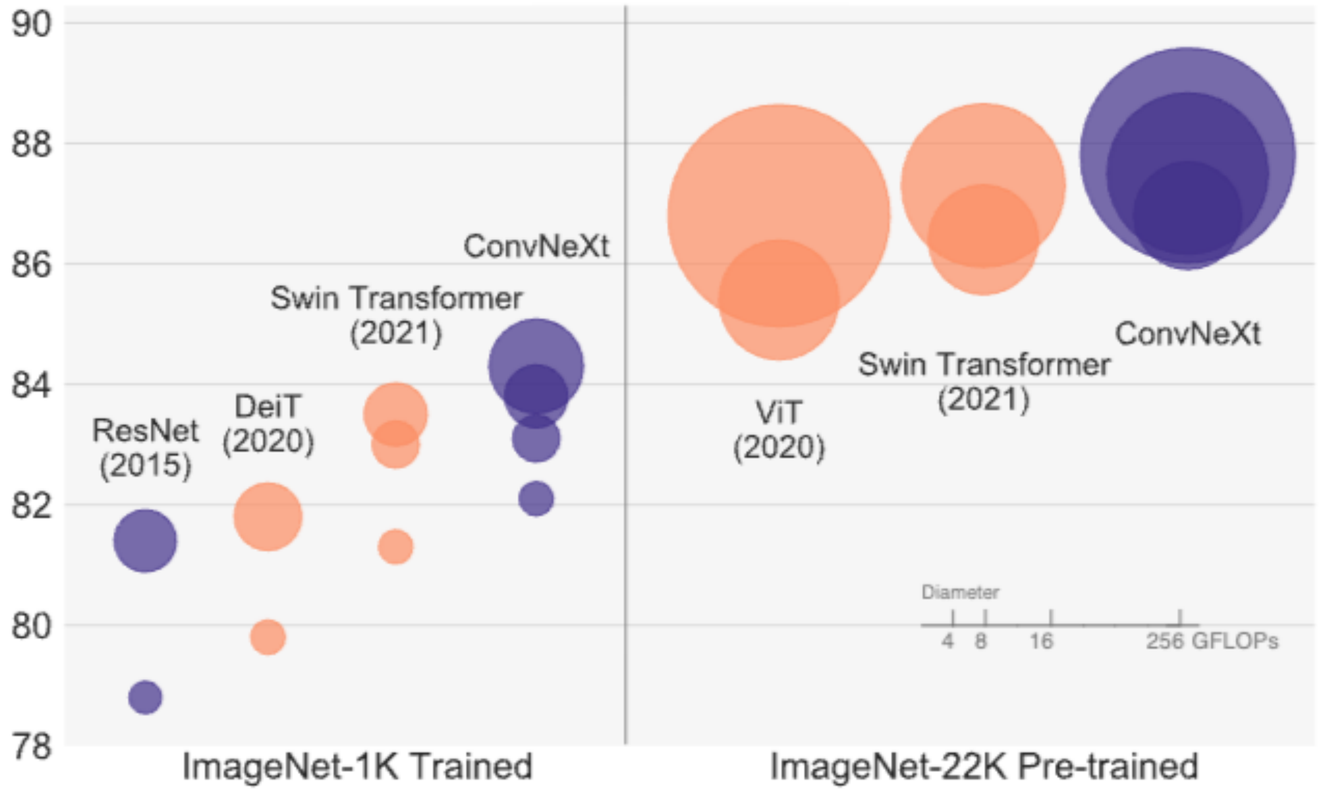
Transformer가 강해진 시대,
CNN도 현대적으로 다시 설계하면
경쟁력이 생기지 않을까?

→

ResNet을 출발점으로
순수 CNN 구조를 현대적으로 재설계

ConvNeXt

convNeXt 성능 비교



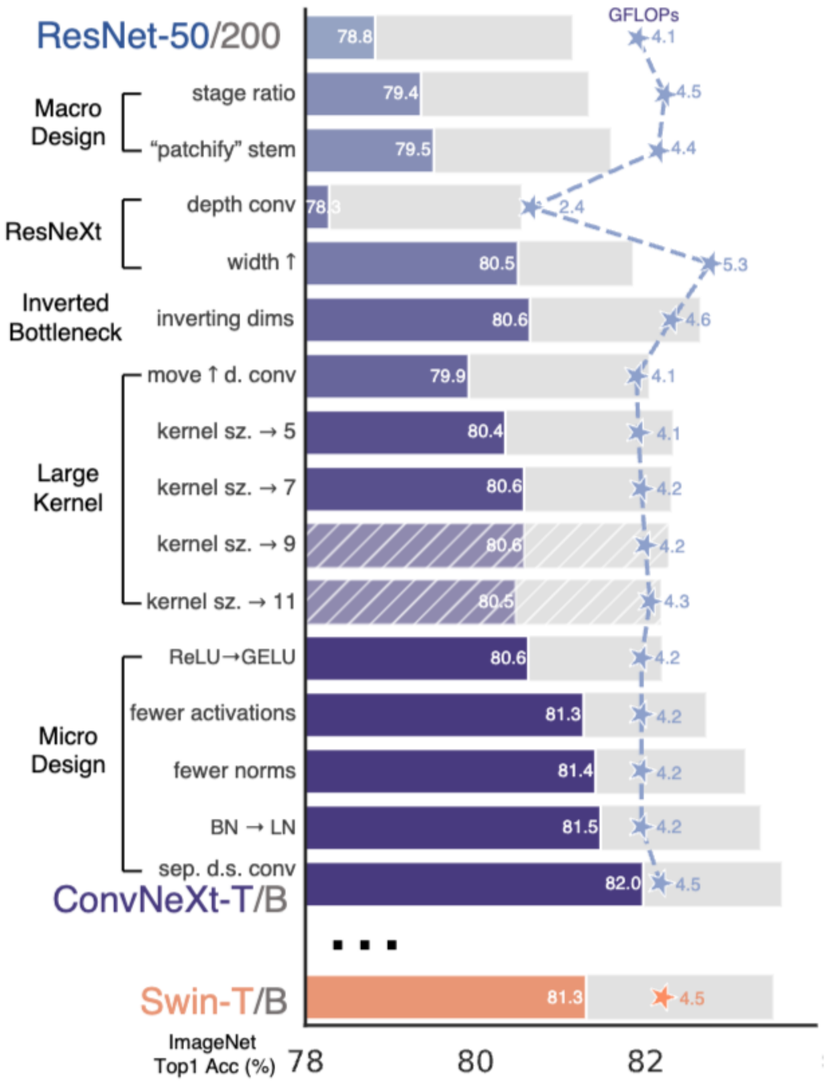
ConvNeXt

ResNet의 현대화 과정

- 기존 ResNet-50 모델이 아닌 ViT를 훈련하는 데 사용된 유사한 훈련 기법으로 훈련한 ResNet-50 모델 사용

  76.1% -> 78.8% (+2.8%)

- 모든 모델은 ImageNet-1K에서 훈련 및 평가

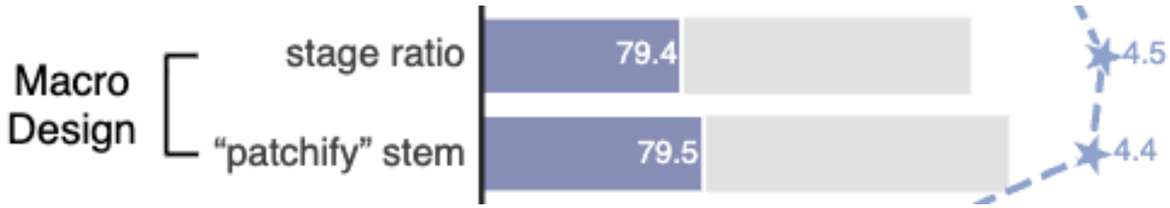


ConvNeXt

Macro Design

- Stage ratio : 각 스테이지의 블록 수를 1:1:3:1 비율로 맞춰 Swin Transformer의 구조와 맞춤
  -> FLOPs 동일

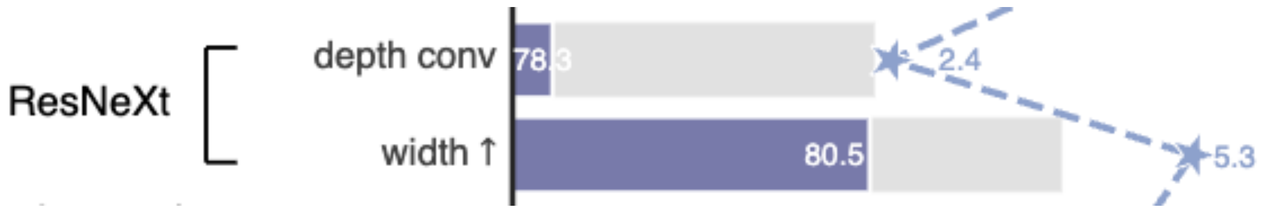
- Patchify stem : Swin Transformer 처럼 처음부터 이미지를 4x4 patch 단위로 나누는 방식 -> 4x4 Conv, stride 4



ConvNeXt

ResNeXt-ify

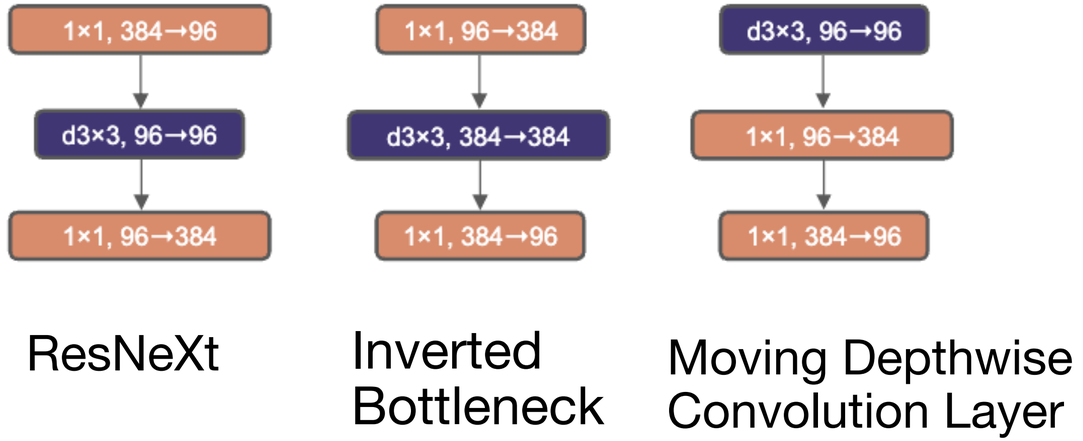
- Depthwise convolution : 그룹 수가 채널 수와 같은 그룹화된 convolution
- Depthwise Conv : 공간 정보 혼합 (Self-attention)
- 1x1 Conv : 채널 정보 혼합 (MLP)
- 너비 (채널 수)를 96으로 늘려 FLOPs와 정확도 늘림



ConvNeXt

Inverted Bottleneck

- Transformer 구조와 같이 역병목 구조로 설계
  -> Depthwise conv에서 FLOPs 증가
  But, 1x1 conv 에서 FLOPs 크게 감소





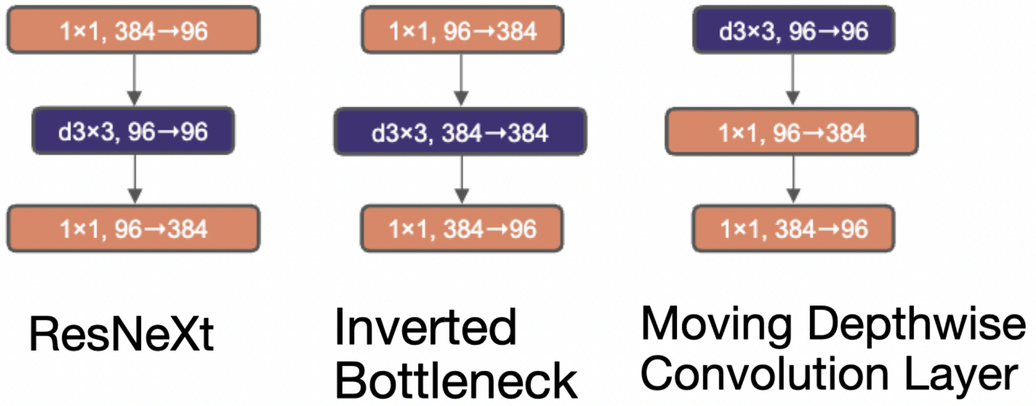
ConvNeXt

Moving up depthwise conv layer

- Depthwise conv 를 block 앞쪽으로 이동

- 계산 부담 ↓, FLOPs ↓, 성능 ↓

성능을 향상 시키려면?

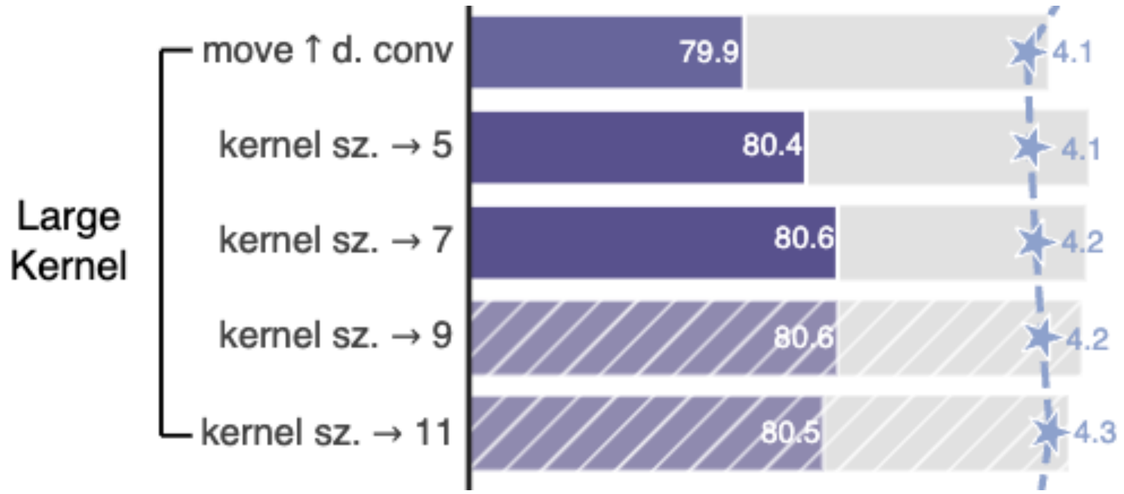


ConvNeXt

Large Kernel Sizes

- 더 큰 커널 크기의 conv 채택
- 커널 크기 7x7일 때 최대 성능

  -> 이때 네트워크의 FLOPs는 대략 동일



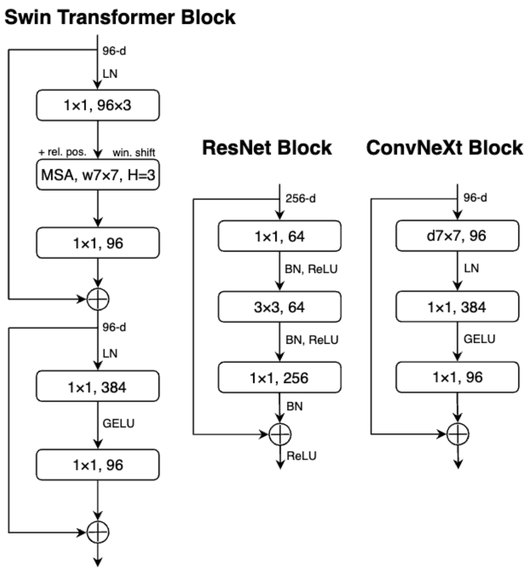
ConvNeXt

Micro Design (미시적 관점)

- 활성화 함수

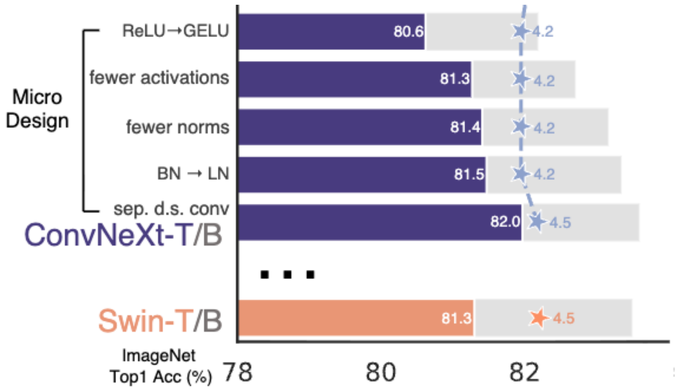
-ReLU -> GELU 변환 (정확도 변화 x)

- 적은 활성화 함수
- 적은 정규화 레이어
- BN -> LN 변환



	output size	• ResNet-50	• ConvNeXt-T	○ Swin-T
stem	56×56	7×7, 64, stride 2 3×3 max pool, stride 2	4×4, 96, stride 4	4×4, 96, stride 4
res2	56×56	[1×1, 64 3×3, 64 1×1, 256] × 3	[d7×7, 96 1×1, 384 1×1, 96] × 3	[1×1, 96×3 MSA, w7×7, H=3, rel. pos. 1×1, 96 1×1, 384 1×1, 96] × 2
res3	28×28	[1×1, 128 3×3, 128 1×1, 512] × 4	[d7×7, 192 1×1, 768 1×1, 192] × 3	[1×1, 192×3 MSA, w7×7, H=6, rel. pos. 1×1, 192 1×1, 768 1×1, 192] × 2
res4	14×14	[1×1, 256 3×3, 256 1×1, 1024] × 6	[d7×7, 384 1×1, 1536 1×1, 384] × 9	[1×1, 384×3 MSA, w7×7, H=12, rel. pos. 1×1, 384 1×1, 1536 1×1, 384] × 6
res5	7×7	[1×1, 512 3×3, 512 1×1, 2048] × 3	[d7×7, 768 1×1, 3072 1×1, 768] × 3	[1×1, 768×3 MSA, w7×7, H=24, rel. pos. 1×1, 768 1×1, 3072 1×1, 768] × 2
FLOPs		4.1×10^9	4.5×10^9	4.5×10^9
# params.		25.6×10^6	28.6×10^6	28.3×10^6

Block 구조와 stage별 네트워크 전체 구조



ConvNeXt

Separate downsampling layers

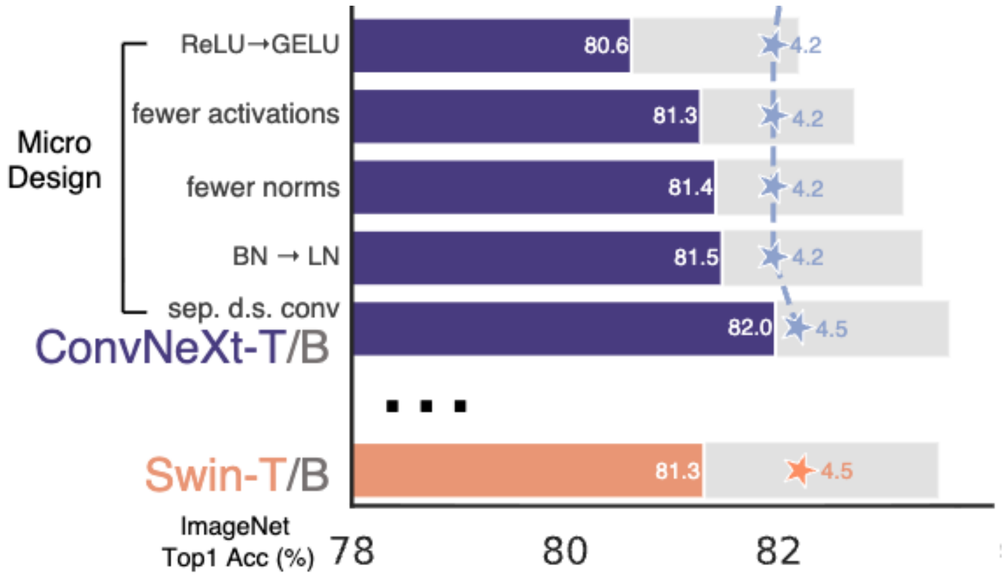
- 2x2 Conv, Stride 2 다운 샘플링 레이어를 stage 사이에 넣는 방법
  (Swin Transformer의 patch Merging)->
  But, 학습 발산

- LayerNorm(LN) 추가

  - 각 downsampling layer 앞

  - Stem 뒤

  - 최종 gobal average pooling 뒤



ConvNeXt

Experiments (background)

ConvNeXt-T: $C = (96, 192, 384, 768), B = (3, 3, 9, 3)$
ConvNeXt-S: $C = (96, 192, 384, 768), B = (3, 3, 27, 3)$
ConvNeXt-B: $C = (128, 256, 512, 1024), B = (3, 3, 27, 3)$
ConvNeXt-L: $C = (192, 384, 768, 1536), B = (3, 3, 27, 3)$
ConvNeXt-XL: $C = (256, 512, 1024, 2048), B = (3, 3, 27, 3)$

- ConvNeXt-T/B 는 각각 ResNet-50/200 (현대화)
- C : 채널 수
- B : 각 stage의 block 수
- ImageNet-1K 와 ImageNet-22K 데이터셋 이용

ConvNeXt

ImageNet-1K, ImageNet-22K

- 이전 최신 CNN 과 최신 Transformer 모델들과 ConvNeXt 의 성능 비교
- ConvNeXt가 비교적 좋은 성능
- VIT가 귀납적 편향이 적어 좋을것이다 라는 인식

  But, ConvNeXt가 동등하거나 더 높은 성능

model	image size	#param.	FLOPs	throughput (image / s)	IN-1K top-1 acc.
ImageNet-1K trained models					
● RegNetY-16G [54]	224^2	84M	16.0G	334.7	82.9
● EffNet-B7 [71]	600^2	66M	37.0G	55.1	84.3
● EffNetV2-L [72]	480^2	120M	53.0G	83.7	85.7
○ DeiT-S [73]	224^2	22M	4.6G	978.5	79.8
○ DeiT-B [73]	224^2	87M	17.6G	302.1	81.8
○ Swin-T	224^2	28M	4.5G	757.9	81.3
● ConvNeXt-T	224^2	29M	4.5G	774.7	**82.1**
○ Swin-S	224^2	50M	8.7G	436.7	83.0
● ConvNeXt-S	224^2	50M	8.7G	447.1	**83.1**
○ Swin-B	224^2	88M	15.4G	286.6	83.5
● ConvNeXt-B	224^2	89M	15.4G	292.1	**83.8**
○ Swin-B	384^2	88M	47.1G	85.1	84.5
● ConvNeXt-B	384^2	89M	45.0G	95.7	**85.1**
● ConvNeXt-L	224^2	198M	34.4G	146.8	**84.3**
● ConvNeXt-L	384^2	198M	101.0G	50.4	**85.5**
ImageNet-22K pre-trained models					
● R-101x3 [39]	384^2	388M	204.6G	-	84.4
● R-152x4 [39]	480^2	937M	840.5G	-	85.4
● EffNetV2-L [72]	480^2	120M	53.0G	83.7	86.8
● EffNetV2-XL [72]	480^2	208M	94.0G	56.5	87.3
○ ViT-B/16 (☎) [67]	384^2	87M	55.5G	93.1	85.4
○ ViT-L/16 (☎) [67]	384^2	305M	191.1G	28.5	86.8
● ConvNeXt-T	224^2	29M	4.5G	774.7	**82.9**
● ConvNeXt-T	384^2	29M	13.1G	282.8	**84.1**
● ConvNeXt-S	224^2	50M	8.7G	447.1	**84.6**
● ConvNeXt-S	384^2	50M	25.5G	163.5	**85.8**
○ Swin-B	224^2	88M	15.4G	286.6	85.2
● ConvNeXt-B	224^2	89M	15.4G	292.1	**85.8**
○ Swin-B	384^2	88M	47.0G	85.1	86.4
● ConvNeXt-B	384^2	89M	45.1G	95.7	**86.8**
○ Swin-L	224^2	197M	34.5G	145.0	86.3
● ConvNeXt-L	224^2	198M	34.4G	146.8	**86.6**
○ Swin-L	384^2	197M	103.9G	46.0	87.3
● ConvNeXt-L	384^2	198M	101.0G	50.4	**87.5**
● ConvNeXt-XL	224^2	350M	60.9G	89.3	**87.0**
● ConvNeXt-XL	384^2	350M	179.0G	30.2	**87.8**

ConvNeXt

Isotropic ConvNext vs ViT

- ConvNext가 ViT 처럼 중간 다운샘플링 없이 해상도를 유지하는 등방성 구조에서 성능 비교

- 동등하거나 더 나은 성능 달성

model	#param.	FLOPs	throughput (image / s)	training mem. (GB)	IN-1K acc.
○ ViT-S	22M	4.6G	978.5	4.9	79.8
● ConvNeXt-S (*iso.*)	22M	4.3G	1038.7	4.2	79.7
○ ViT-B	87M	17.6G	302.1	9.1	81.8
● ConvNeXt-B (*iso.*)	87M	16.9G	320.1	7.7	82.0
○ ViT-L	304M	61.6G	93.1	22.5	82.6
● ConvNeXt-L (*iso.*)	306M	59.7G	94.4	20.4	82.6

ConvNeXt

COCO object detection

- COCO 데이터셋에서의 객체 탐지, 인스턴스 분할 성능 비교
- 동등하거나 더 나은 성능 달성

backbone	FLOPs	FPS	AP^{box}	AP^{box}_{50}	AP^{box}_{75}	AP^{mask}	AP^{mask}_{50}	AP^{mask}_{75}
Mask-RCNN 3× schedule								
○ Swin-T	267G	23.1	46.0	68.1	50.3	41.6	65.1	44.9
● ConvNeXt-T	262G	25.6	**46.2**	67.9	50.8	**41.7**	65.0	44.9
Cascade Mask-RCNN 3× schedule								
● ResNet-50	739G	16.2	46.3	64.3	50.5	40.1	61.7	43.4
● X101-32	819G	13.8	48.1	66.5	52.4	41.6	63.9	45.2
● X101-64	972G	12.6	48.3	66.4	52.3	41.7	64.0	45.1
○ Swin-T	745G	12.2	50.4	69.2	54.7	43.7	66.6	47.3
● ConvNeXt-T	741G	13.5	**50.4**	69.1	54.8	**43.7**	66.5	47.3
○ Swin-S	838G	11.4	51.9	70.7	56.3	45.0	68.2	48.8
● ConvNeXt-S	827G	12.0	**51.9**	70.8	56.5	**45.0**	68.4	49.1
○ Swin-B	982G	10.7	51.9	70.5	56.4	45.0	68.1	48.9
● ConvNeXt-B	964G	11.4	**52.7**	71.3	57.2	**45.6**	68.9	49.5
○ Swin-B‡	982G	10.7	53.0	71.8	57.5	45.8	69.4	49.7
● ConvNeXt-B‡	964G	11.5	**54.0**	73.1	58.8	**46.9**	70.6	51.3
○ Swin-L‡	1382G	9.2	53.9	72.4	58.8	46.7	70.1	50.8
● ConvNeXt-L‡	1354G	10.0	**54.8**	73.8	59.8	**47.6**	71.3	51.7
● ConvNeXt-XL‡	1898G	8.6	**55.2**	74.2	59.9	**47.7**	71.6	52.2

ConvNeXt

Semantic segmentation on ADE20K

- ADE20K 데이터셋에서의 의미론적 분할 성능 비교
- 동등하거나 더 나은 성능 달성

backbone	input crop.	mIoU	#param.	FLOPs
ImageNet-1K pre-trained				
○ Swin-T	512^2	45.8	60M	945G
● ConvNeXt-T	512^2	**46.7**	60M	939G
○ Swin-S	512^2	49.5	81M	1038G
● ConvNeXt-S	512^2	**49.6**	82M	1027G
○ Swin-B	512^2	49.7	121M	1188G
● ConvNeXt-B	512^2	**49.9**	122M	1170G
ImageNet-22K pre-trained				
○ Swin-B‡	640^2	51.7	121M	1841G
● ConvNeXt-B‡	640^2	**53.1**	122M	1828G
○ Swin-L‡	640^2	53.5	234M	2468G
● ConvNeXt-L‡	640^2	**53.7**	235M	2458G
● ConvNeXt-XL‡	640^2	**54.0**	391M	3335G

코드

```python
import torch
import timm


def test_classification_model(model_name): # 모델을 이미지 분류 모델로 실행했을 때의 입력/출력 크기를 확인하는 함수
    print("=" * 60)
    print(f"[Classification Model] {model_name}")

    # pretrained=True로 하면 사전학습된 모델을 불러옴
    model = timm.create_model(model_name, pretrained=True)
    model.eval()

    # 더미 입력 이미지 생성
    # shape: [batch_size, channel, height, width]
    x = torch.randn(1, 3, 224, 224)

    # 모델 실행
    with torch.no_grad():
        y = model(x)

    print("Input shape :", x.shape)
    print("Output shape:", y.shape)
    print("Meaning     : ImageNet 1000 classes에 대한 분류 점수")
    print()
```

```python
  def test_backbone_features(model_name):#모델을 백본으로 사용했을 때 stage별 feature map 크기를 확인하는 함수
      print("=" * 60)
      print(f"[Backbone Feature Extractor] {model_name}")

      # features_only=True를 사용하면 최종 분류 결과가 아니라
      # stage별 feature map을 출력함
      model = timm.create_model(
          model_name,
          pretrained=True,
          features_only=True
      )
      model.eval()

      x = torch.randn(1, 3, 224, 224)

      with torch.no_grad():
          features = model(x)

      print("Input shape:", x.shape)

      for i, feature in enumerate(features):
          print(f"Stage {i + 1} feature shape:", feature.shape)

      print("Meaning: 각 stage에서 추출된 feature map")
      print()
```

```
============================================================
[Classification Model] swin_tiny_patch4_window7_224
Input shape : torch.Size([1, 3, 224, 224])
Output shape: torch.Size([1, 1000])
Meaning     : ImageNet 1000 classes에 대한 분류 점수

============================================================
[Classification Model] convnext_tiny
Input shape : torch.Size([1, 3, 224, 224])
Output shape: torch.Size([1, 1000])
Meaning     : ImageNet 1000 classes에 대한 분류 점수

============================================================
[Backbone Feature Extractor] swin_tiny_patch4_window7_224
Input shape: torch.Size([1, 3, 224, 224])
Stage 1 feature shape: torch.Size([1, 56, 56, 96])
Stage 2 feature shape: torch.Size([1, 28, 28, 192])
Stage 3 feature shape: torch.Size([1, 14, 14, 384])
Stage 4 feature shape: torch.Size([1, 7, 7, 768])
Meaning: 각 stage에서 추출된 feature map

============================================================
[Backbone Feature Extractor] convnext_tiny
Input shape: torch.Size([1, 3, 224, 224])
Stage 1 feature shape: torch.Size([1, 96, 56, 56])
Stage 2 feature shape: torch.Size([1, 192, 28, 28])
Stage 3 feature shape: torch.Size([1, 384, 14, 14])
Stage 4 feature shape: torch.Size([1, 768, 7, 7])
Meaning: 각 stage에서 추출된 feature map
```

References

- [1] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, Baining Guo,

  “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows,” ICCV, 2021.

- [2] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, Saining Xie,

  “A ConvNet for the 2020s,” CVPR, 2022.

감사합니다!